

Assim de
Borden, prevenindo
Safety and Security.

ação não
intencional
de pessoa ou
Brenini perder em
relação a pessoa
mais intencional



Requisitos de Software: É uma propriedade/capacidade que deve estar presente em um sistema e/ou software para resolver um problema do mundo real.

↳ Requisito de Produto: Capacidades que devem estar no software para atender demanda

↳ Requisito de Processo: Tempo, Escopo e custo : Geralmente.

↳ Requisito Funcional: Algoritmo, independente de tecnologia

↳ Requisito não funcional: Determinam o quão aquela solução atende aos requisitos funcionais.

↳ Sabe-se que você tem o pc perfeito, quando você faz um sistema acadêmico. Você ainda precisa que um aluno pague uma matrícula (Requisito funcional) ↳ (Requisito Não Funcional)

Elicitação de Requisitos: Geralmente onde pode ser visto requisitos são dos próprios Stakeholders, analisar apps parecidas, banco de dados com erros, você mesmo, literatura

Análise de Requisitos: Não deve ser ambíguo um requisito, deve ser testável, o requisito tem que agregar valor, usar vocabulário dos stakeholders, ser algo que realmente precise, verificável

↳ 5 whys → Analisa realmente se o requisito é ou não importante.

↳ Necessário saber quando um requisito entra em conflito entre stakeholders, o que fazer geralmente é procurar um "meio-termo"

Especificação dos requisitos: Usando linguagem natural a documentação pode ser estruturada ou não estruturada. Regras de Negócio.

ATD (ITD) = técnica de elaboração de acordo os requisitos, primeiro são ^{elaborados} ~~requisitos~~ e ^{no teste} ~~requisitos~~ para realmente depois criar uma solução. Quando a solução for aprovada, ^{no teste} ~~requisitos~~ ^{requisitos} para encontrar uma melhor solução.

BDD = Maior quantidade possível de testes para verificar se os requisitos foram realmente preenchidos correlacionando. (entenda/demanda por teste automaticamente) ↳ pensando se atendeu a todos

Validação de Requisitos: Tem haver com realmente assegurar que aqueles requisitos são realmente o que os stakeholders querem

↳ Revisão de requisitos, simulação e execução e prototipagem



Controle de Alteração de Requisitos

Em método de casuário é um pouco mais complicado de se realizar qualquer alteração. Já em método ágil, eles são feitos pensando nisso, ou seja, a mudança é bem vinda.

• Alterar escopo, e outras maneiras são formas.

→ Priorizar as funcionalidades é essencial para um bom projeto.

Considerações Práticas: O tratamento de requisitos deve ser algo feito ao longo do projeto, dificilmente você conseguirá ~~elaborar tudo de~~ ^{elaborar tudo de} uma vez.

Como priorizar?: Se perguntar quais fatores farão uma atividade ser mais importante que outra primeiro, pelo valor ao cliente, maior risco. O GRUPO DEFINI

→ Existem estudos que tentam quantificar a prioridade, através de estudos, formulas

Rastreabilidade é muito importante dentro da documentação.

Requisitos podem ser estáveis/instáveis.

Normalizar requisitos: é uma maneira de calcular tempo/ponto/esforço que facilita a obter conclusões sobre determinado requisito

→ Indicadores de qualidade: comparações (benchmark), ex. b1, ...

Ferramentas de Software de requisitos: Facilitam muito ao se tratar de grande empresas, mas a questão maior é não faz sentido para projetos grandes usar coisas feitas com o um backlog, até mesmo pelos riscos

Engenharia de Sistemas e as normas de Certificação INCOSE

Definição

Definição: Arranjo de partes ou elementos que exibem coletivamente ~~exibem~~ ^{exibem} o comportamento que as constituintes não exibem - É mais que as somas das partes.

Concept Development → Requirements Engineering → System Architecture Development →

System Design and Development → System Integration → System test and Evaluation →

Manufacturing → Operation, Maintenance and System Evaluation

INCOSE: International Council On Systems Engineering

→ ASEP: Profissional de Engenharia de Sistemas Associado

→ CSEP: Profissional de Engenharia de Sistemas Certificado

tilibra → ESEP: Profissional de Engenharia de Sistemas Expert

provas de certificação.

Engenharia de Requisitos no contexto de Engenharia de ^{Sistema} ~~Software~~ (Parte 1 de 2)

Necessidade e Requisito: Necessidade são capacidades que integram um nível de negócio. Requisitos são derivados de necessidades, entretanto mais formais, estruturados que podem ser validados.

Conceitos de ciclo de vida:

Operations (Ops/Con): Descreve o que sistema vai fazer, como e porque. Da perspectiva do próprio usuário.

Acquisition: Descreve o caminho para que o sistema seja adquirido.

↳ engajamento stakeholders, design, produção, verificação, requisitos.

Deployment: Implantação no ambiente de produção, + define como nova entrega, validade e introduzido as operações.

Support: Descreve a infraestrutura de suporte desejada e mão de obra para que se possa dar suporte/manutenção depois de o sistema estar entregue.

Retirement: Descreve o caminho de como o sistema será tirado do ar e aposentado.

Functional and Non-functional Requirements

↳ Functional: O que o sistema deve fazer, algoritmos etc.

↳ Non functional: As qualidades das coisas, performance, escalabilidade, beleza...

↳ Rationale Statements

• User: Qualquer um que esteja usando o sistema desde que ele foi desenvolvido.

• Customer: Organizações que pagam para o sistema ser desenvolvido.

• Acquirer: Entidade que irá obter o produto.

• Developer: Quem transforma os requisitos no sistema.

• Contractor: Organizações que ficaram responsáveis pelo desenvolvimento do produto contratado por terceiros.

Características que um requisito deve possuir: Necessary, Singular, Correct, Unambiguous, feasible, appropriate to level, complete, conforming, verifiable.

Atributos que um requisito deveria ter: Identifier, unique short title, priority, risk, source.

Lista de Exercícios 2

~~Ex 1 - O que é a importância de requisitos e como eles influenciam o projeto de software?~~

Ex-2. O SWEBOX destaca que requisitos podem ser incompletos ou ambíguos. Explique esses dois problemas e como podem afetar um projeto de software.

Basicamente, a ambiguidade pode fazer com que realmente o cliente queira, tenha um resultado diferente do que foi realmente proposto, pois o requisito inicialmente tem mais de uma ideia. Já o fato de ser incompleto tem mais a ver com o fato de o que foi implementado também ficar incompleto, fazendo com que o trabalho seja redobrado até se completar o requisito.

EX3- Compare a contraste requisitos funcionais / não funcionais, fornecendo exemplos.

O requisito funcional está ligado com aquela tipo funcionalidade que é imprescindível para o funcionamento correto do sistema, o que deve realmente ser feito. → Algoritmos essenciais.

Já o não funcional está relacionado ao que não é necessário para que o sistema realmente funcione, mas é bem mais: estética, desempenho e afins.

EX4- Explique o que caracteriza o modelo de ciclo de vida "V" na engenharia de sistemas e sua aplicabilidade prática.

O que caracteriza o modelo de ciclo de vida "V" é: uma diagrama que representa as etapas de desenvolvimento de um sistema de forma visual, formado "V" invertido. Organizado desde a fase de concepção até as operações e manutenção, sendo utilizado para validar projetos e processos.

EX5- Qual é a importância do INCOSE para a área de Engenharia de Sistemas?

O INCOSE é o conselho internacional de Engenharia de Sistemas, a mais principal para a área de Engenharia de sistemas são para certificação profissional dentro da área de engenharia de sistemas, e também é quem define as normas e padrões **tilibra** o desenvolvimento da engenharia de sistemas, organizando o trabalho.

Uma hierarquia de requisitos é produzida durante: elicitação e elaboração.

Elaboração: ocorre a quebra de requisitos de nível alto em requisitos menores.

↳ Junto da elicitação não é algo que deve ser feito por novatos, mas sim, por pessoas experientes que entendam do negócio.

Verificação e validação

↳ Verificação: No meio da produção do sistema, você verifica se o que está sendo feito, está sendo feito da maneira correta. "Construir certo a coisa"

↳ Validação: Pegar o produto final e analisar se ele está conforme o requisito que o stakeholder propuseram. "Construir a coisa certa"

Teste e Evolução

↳ Os testes não são mais feitos apenas no final do produto, podemos dizer que cada fase de produção é necessário ter um teste. ("Quanto antes detectar, mais fácil resolver")

Mission → Goals → Objectives → Requerimentos L1 → Requerimentos L2.

Restrições

↳ Fornecer de onde o requisito veio, qual é a relação entre o requisito, quais são as implementações / que foram derivadas desse sistema.

Lacunas de capacidade

↳ É necessário pensar estrategicamente e operacionalmente. Pensando não só em atender a curto prazo, mas também um planejamento a longo prazo.

↳ Porém podem existir algumas lacunas durante o planejamento a longo prazo,

↳ E essas lacunas atrapalham principalmente quando estamos tratando sobre o que pode ser o ~~resultado~~ o resultado do planejamento, mas as lacunas atuais podem impedir

Exemplo: "Quero surfar em Porto Seguro"

↳ Lacunas: "Você tem uma prancha", "Você sabe surfar", "Você está em Porto Seguro"

Ao encontrar "gaps", automaticamente conseguimos encontrar "need" que precisam ser realizadas para que aquilo deixe de ser uma lacuna.

Design Conceitual: Define a necessidade / requisitos do negócio, stakeholder, sistema, conduzir a análise em nível sistema, revisão do Design do Sistema.

Síntese x Design: Síntese é um aspecto crítico do processo de design que foca mais na arquitetura de alto nível e na integração do sistema. Design: Além de síntese, especificação detalhada, implementação, validação de elementos. Síntese é um passo chave dentro do Design.

Exercício 1 - Explique como a rastreabilidade de requisitos pode auxiliar no impacto de mudanças em projetos de software.

Basicamente, quando algo está claramente rastreável é muito mais fácil de encontrar onde tal erro pode estar ocorrendo, onde procurar por cada tipo de coisa, auxiliar na documentação. Mas o principal benefício mesmo é saber onde está aquilo que você deseja modificar.

Exercício 2 - Métodos como ATDD e BDD são usados para reduzir ambiguidade no requisitos. Escolha um deles e explique como funciona.

O método ATDD, diferentemente da ordem usual na qual os programadores são feitos, a partir da análise dos requisitos, são criadas todas as tarefas possíveis a partir deles. A partir disso, é necessário ir criando uma solução para os requisitos (não necessariamente a melhor), assim que essa solução for aprovada nos respectivos testes você pode melhorar essa solução.

Exercício 3 - Quais são os principais desafios da elicitação de requisitos? Cite e explique pelo menos três!

Os principais desafios ao elicitar requisitos são: não deixar com que um requisito seja ambíguo, verificar a real necessidade desse requisito, saber utilizar a mesma linguagem da stakeholder, priorizar os requisitos, evitar requisitos conflitantes.

Requisitos Suficientes: Realmente verificar se os requisitos criados são o suficiente para que o que foi pedido seja realmente produzido, ou seja podem estar faltando requisitos.

Ambiguidade: É de extrema importância deixar os requisitos o mais claros possíveis de acordo a real ideia do produto, tendo em vista que a ambiguidade pode atrasar e muito o andamento, já que algo que não era para ser implementado pode ser.

Priorizar Requisitos: Saber o que deve ser implementado primeiro para que o MVP do projeto seja realmente feito. Então, saber a ordem em que cada coisa deve ser feita auxilia muito em estar ligado com a ideia do projeto.

Exercício 4 - Quais são as diferenças entre Software e Sistema?

Podemos perceber diferenças claras entre Software e Sistema, quando o sistema engloba além de software, hardware, pessoas, processos institucionais, legislação, todos trabalhando de forma pelo propósito do sistema.

O software, entretanto, é uma aplicação de sistema operacional onde reuni alguma rotina e mais com o propósito único de rodar o software em um dispositivo.

Exercício 5 - Quais são as semelhanças entre Sistema e Software?

A semelhança que consigo perceber logo de início é que em ambas são necessárias uma análise de requisitos, formalização de organização para a produção de, tanto o software quanto o sistema não possuem com a intenção de resolver um problema, tanto um quanto o outro se modificam ao decorrer do tempo, possuem um ciclo de vida.

Exercício 6 - A área de Engenharia de Sistemas também usa o conceito de requisitos?

Sim, tanto a engenharia de sistema/software utilizam do conceito de requisitos. Para o levantamento de requisitos definimos: O que deve ser feito, como deve ser feito (no sentido de o que deve ser feito para o que l ser respondido), o que não deve ser feito, quais restrições são impostas, portanto o ciclo de vida de sistema/software depende e muito do levantamento dos requisitos.

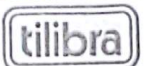
Exercício 7 - Na área de Engenharia de Sistemas, em nível de negócios/stakeholders, o termo "necessidade" é bastante difundido. Qual a relação entre Business needs / necessidades de negócios, Stakeholders needs e Requirements?

Basicamente, em um geral (necessidade e requisitos) são a mesma coisa. Entretanto, requisitos são tanto as (Business / Stakeholders) needs de uma maneira mais formal, estruturada e de uma forma em que eles podem ser ou não validados.

Business: Definem estratégia e motivação da organização.

Stakeholders: Repletos as necessidades específicas de todos os envolvidos no sistema.

Requisitos: São as especificações detalhadas de como o sistema atenderá essas necessidades.



STPA = System Theoretic Process Analysis

O que é um sistema: Amontado de componentes que trabalham juntos por um objetivo comum. → **Objetivo é fundamental**

↳ Cada observador enxerga diferente propósito em um mesmo sistema.

Documentação é essencial, onde criamos especificações do sistema: Objetivo do sistema.

* **Engineered Systems**: Objetivo é criado antes do sistema, mesmo com diferentes visões de design.

Natural Systems: O comportamento não é previsto, o comportamento é observado.

Sistema é recursivo: Pode ser parte de um sistema maior, ou subdividido em parte.

* **Open System**: Interações com o mundo exterior

Closed Systems: Não interagem com o mundo exterior

Engenharia de Sistemas: Tentativa de colocar uma estrutura tanto no design quanto na construção de sistemas com o objetivo de aperfeiçoar o esforço de engenharia.

↳ Modelo mais famoso dentro da engenharia de sistemas - **V-MODEL**

↳ Pode-se imaginar como uma diretriz, apenas incluindo a existência de um processo que vai gerar o desenvolvimento do sistema. → **Guiado pela documentação**

↳ Uma especificação preliminar tem: objetivos, propriedades relacionadas ao interesse do sistema, como os objetivos devem ser atingidos, o limite do sistema também - **sempre refinado**,

O **limite do sistema**: Deve ser identificado na especificação, onde se define o que deve estar dentro do controle do designer e o que não está. → **O que está fora do sistema?**

Controle: Está presente conception, design and operation, sem o controle é muito difícil

assegurar que os objetivos do sistema sejam satisfeitos. **processos**: controle de design, processo, social

Design Control: Para que este tipo de controle melhore o safety sem aumentar a complexidade, é necessário ele ser implementado desde o início do projeto.

↳ usar um design que elimine ^{um design} hazards, que seja mais a prevenção de hazards, um design que minimize os efeitos de hazards.

↳ **Técnica Intrínseca**: basicamente achar um meio que é impossível de ocorrer. ^{para uma análise de um sistema}

↳ **Técnica Passiva**: implementar ações que são feitas disponíveis mas que podem ser feitas ^{maquina controla alguma coisa}

↳ **Técnica Ativa**: basicamente é quando já ocorreu o hazard, e a partir daí alguma ação é tomada a partir daí.

O que é um control loop?: Pode ser usado em várias ocasiões // Processo

→ Controller: Manter o sistema dentro dos limites pré-estabelecidos de segurança.

→ Control Actions: Implementadas pelo atuador, ^{de acordo com o processo} implementação da control action e para manter o sistema dentro dos limites de segurança.

→ Feedback: Implementado a partir de sensores, que detectam o que é preciso.

→ Process Model: visão do estado do sistema que o controlador tem

→ Control Algorithm: Representa a tomada de decisão sobre qual ação deve ser tomada.

→ Controlled Process: É o processo que está sendo controlado por um controlador ^{muda seu estado a partir das control actions}

Feedback Operational Model: Baseado em um feedback, em torno de tomadas de decisão.

Feedforward Control: Pega o feedback atual + visão do futuro para que se tome uma decisão

*1 Teoria de Sistemas: STARP baseada e não a teoria de complexidade = conjunto de princípios que pode ser utilizado para se entender sistemas complexos

2 Teoria da Complexidade: baseia a partir da teoria de sistemas e outros conceitos de 60.

1 x 2 = Emergência, hierarquia, comunicação e controle. Ambos focam no comportamento do sistema complexo como um todo, os dois se focam na redução/decomposição

1 = Toda o sistema possuem propriedades emergentes // ^{emergentes} Mais apropriado para sistemas

2 = Divide sistema em: simples, complicados, complexos e caóticos // ^{diversos} Diferentes Naturezas

01 - Há diferença em como requisitos evoluem de acordo com um modelo de ciclo de vida de processos da engenharia de software e engenharia de sistemas? Sim, existem diferenças de como os requisitos evoluem de acordo com cada tipo de engenharia. Mesmo que ambas possuam princípios parecidos, como gerenciar e validar requisitos ao longo do tempo, a maneira como isso é feito é diferente em cada uma, onde geralmente a engenharia de software é mais ligada a métodos ágeis, já a engenharia de sistemas é mais adaptada a sistemas mais rígidos. Tendo em vista que qualquer mudança é muito mais difícil na realização de qualquer mudança, até mesmo quando estamos falando de modelos existentes, já existem diferenças.

2) Explique a diferença entre o domínio do problema e o domínio da solução em engenharia de sistemas?
 Porquê? Como?
 descrição lógica/funcional descrição física

A diferença desses domínios é: o domínio do problema trabalha basicamente com o que é um problema, e como esse problema afeta os stakeholders. Já o domínio da solução trabalha em como esse problema realmente irá ser resolvido.

3) Descreva como as necessidades dos stakeholders são transformadas em engenharia de sistemas → análise requisitos, negócios e mídias.

Elas são transformadas por meio de algumas fases: elicitação, análise e elaboração formalizadas e validadas. Isso tudo faz com que muitas palavras dos stakeholders possam se tornar coisas mais técnicas e fáceis de serem elaboradas.

4) Quais são os principais documentos ou artefatos produzidos durante o processo de análise de requisitos? Explique brevemente a função de cada um.

Business Requirements Specification (BRS): "Documenta os requisitos de negócios derivados das necessidades da gestão de negócios."

System Requirements Specification (SRS): "Detalha os requisitos dos stakeholders, geralmente relacionados às necessidades operacionais do negócio, para serem validados e implementados."

05- Explique por que a engenharia de requisitos é essencial para o desenvolvimento de sistemas.

A engenharia de requisitos é primordial para o desenvolvimento de sistemas, tendo em vista que ela consegue transformar o que a stakeholder querem em definições formais, sem ambiguidades, clara, evitando retrabalho e garantindo que o sistema traballe de forma correta (perante a visão da stakeholder).

06- Quais são as características principais que um requisito deve apresentar?

Não pode ser ambíguo, deve realmente ser útil, claro, e mais independente possível, deve ser viável e de acordo o atual nível do projeto e completo.

07- Qual é a diferença entre cliente, usuário e entidade adquirente no ciclo de vida de um sistema?

Cliente é quem paga pelo desenvolvimento do sistema.

Entidade adquirente: é quem vai que usa implementar o sistema dentro da empresa (não necessariamente o cliente).

Usuário: Qualquer pessoa que usufrui do sistema.

08- O que são requisitos derivados? Exemplifique.

São requisitos que são essenciais para o funcionamento normal do sistema, sendo criados por análise ex-post e não são criados no trabalho pelo stakeholder.

09- Quais são algumas das principais técnicas utilizadas na elaboração de requisitos?

Cite pelo menos e explique como elas contribuem.

Workshop estruturado, brainstorming, entrevistas, protótipos, etc.

Não é possível entender os eventos sem entender o porque esses eventos ocorreram,
↳ 1 cadeia de eventos não é suficiente

1- Descreva o processo hierárquico utilizado para decomposição de requisitos detalhando sua importância no desenvolvimento de sistema e apresentando um exemplo com base em um cenário médico fictício.

O requerimento flui down, começa com a identificação de requisitos de alto nível baseada na visão do sistema. Cada vez mais ela vão gradualmente detalhadas em níveis menores, tomar funcionalidades (epics), US e por último.

2. Explique porque ferramentas de engenharia de requisitos são importantes em sistemas complexos.

As ferramentas de engenharia são importantes pois elas facilitam a documentação de requisitos de forma estruturada, acompanhamento de alterações, rastreabilidade e o principal - ajudando para atingir o objetivo final através de quebrar requisitos.

3. Compare a abordagem de engenharia de requisitos utilizada no Scrum com a abordagem tradicional para sistemas complexos.

No Scrum, os requisitos são gerenciados de forma iterativa e incremental, onde o foco é a priorização e revisão constante em reuniões de planejamento de Sprint. O sistema complexo, a engenharia de requisitos tradicional exige maior formalidade, com documentação detalhada, controle rigoroso de alterações e ferramentas específicas para gerenciar requisitos e interdependências.

4. Explique o conceito de lacuna de capacidade e sua relação com a definição de requisitos em Engenharia de Sistema.

Capability Gap: Refere-se à diferença entre as necessidades identificadas e as capacidades existentes em um sistema. Na engenharia de sistema, a lacuna é usada para derivar os requisitos, que são definidos como capacidades que um sistema precisa ter para atender a necessidade.

Processo de design conceitual transforma necessidades em requisitos documentados.

⑤ Qual é a importância do design conceitual no ciclo de vida de desenvolvimento de sistemas, e como ele está relacionado à Especificação de Requisitos do Sistema.

O design conceitual é fundamental no ciclo de vida de sistema porque permite identificar e transformar as necessidades em requisitos de alto nível.

Modelo de Condições: Modelo que explica como o sistema funciona e avalia como o sistema irá se comportar no futuro.

Modelo de cadeia de eventos: Basicamente, é um modelo onde o evento passado precisa ser concluído para que o próximo realmente possa ser feito. // Eventos sistêmicos não são únicos

→ Existe vários nomes para o mesmo intuito de mensagem.

O conceito de causa raiz era muito desenvolvido como o primeiro evento → Não é.

Estar em causa raiz não é uma coisa boa. → Geralmente sempre há uma falha de componente ou humana (o que de certa maneira não é justo)

Não é possível entender os eventos sem saber o porque deles terem ocorrido

Acidente baseado em falha de componentes: Envolve um ou vários componentes que falham.

→ que pode ser uma falha "aleatória".

Acidente baseado na interação de componentes: Surge falha devido a interação entre componentes, geralmente associada com o grau de acoplamento entre os dados ("complexidade")

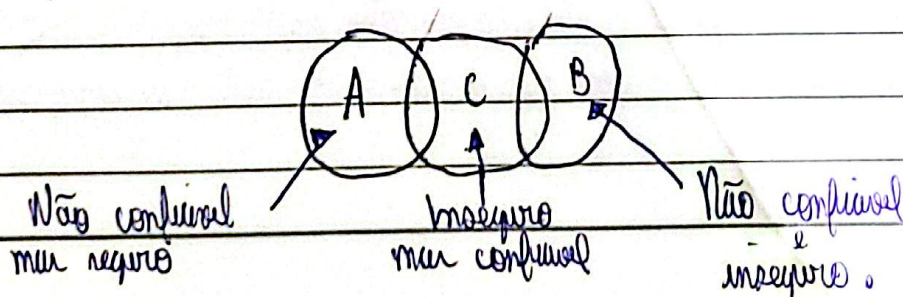
→ Muita problemas associada com o design do sistema.

O conceito de complexidade está criando muitas questões desafiadoras que não podem ser identificadas apenas particionando o sistema em uma cadeia de eventos de falha

O conceito de complexidade aplicado ao sistema atual, está fugindo com que propriedades sistêmicas (Safety and security) que não estão relacionadas com comportamento individual dos sistemas, mas sim relacionadas com a interação entre componentes.

O acidente de hoje não resultam apenas de falha de componentes. Precisamos considerar também erros de design.

Safety and Reliability Scenario



Aspecto do software que mudou o corpo do homem no sistema

- ↳ Software não falha. ①
- ↳ Software permite uma complexidade de sistema quase que ilimitada. ②
- ↳ A maioria dos problemas do software em acidentes geralmente está relacionada à falha na definição de requisitos. ③

1 - O software basicamente um design de máquina abstraído da realidade física.

↳ Sendo puramente design, o design não falha. (O software tem falhas de design)

2 - Tendo em vista toda a complexidade atual dos sistemas, não dá mais a garantir de toda a tipo de erro, dividido... E também é impossível testar tudo.

Contexto determinou se o software é seguro ou não.

O software contribui de maneira diferente a acidentes, do que o hardware.

↳ Geralmente, o problema está identificado em requisitos de safety (errados).

O operador que faz o erro é apenas um sintoma, não há causa.

O erro humano é um sintoma do sistema que precisa de um redesign

Não podemos reduzir efetivamente acidentes sem integrar humanos/software e hardware engenheiros

Nos sistemas antigos, a culpa era sempre do operador, o que tornava a solução tomar alguma ação contra o próprio operador.

↳ Nos sistemas atuais, sabemos que o erro humano é apenas um sintoma.

Resumo Lição Aprendida.

1. É necessário olhar além da aparência para prevenir acidentes.

2. Os acidentes hoje em dia não resultam apenas de falhas de componentes.

↳ Necessário considerar erros em manual de design.

3. O software contribui de maneira diferente para o acidente em comparação ao hardware

↳ O problema principal está relacionado ao levantamento de requisitos.

4. Não podemos atacar safety sem integrar conceitos humanos/software/hardware

Pressupostos de um modelo/teoria que assegurem

↳ Hardware e falhas de hardware ↳ Software (principalmente requisitos)

↳ Fator humano ↳ Interações entre componentes do sistema.

↳ Sistema de forma de design ↳ Administração, manutenção e políciamento.

↳ Fator ambiental é as complexas interações entre elas.

Maneiras de lidar com a complexidade de um sistema

1. Decomposição Analítica 2. Estatística 3. Teoria de Sistema

1 - Dividir sistema em parte, analisando o resultado separadamente e depois o juntando as.

2 - A dependência do sistema estatístico é suficiente, entretanto... não é para tudo

3 - Área descomodável para sistemas muito complexos, porém orgânica

2 - Funciona quando há aleatoriedade suficiente em um sistema, e depende de dados históricos
o que muitas vezes não é possível em novos sistemas

1 - Funciona apenas com relação direta e linear entre os componentes.

3 - O todo é maior do que a soma das partes, onde as propriedades importantes emergem das interações complexas (diretas e indiretas) entre os componentes sistema. Propriedade emergente

podem se parecer com máquinas não funcionais

Mudança paradigma

Segurança para de ser tratada como um problema de confiabilidade e se passa a tratar segurança como um problema de controle. (Mudança para quaisquer máquinas não são

Antes: Foco era prevenir falhas de componentes individuais.

Agora: Foco é impor restrições de segurança a todo o sistema controlado.

Funcionamento do Controle por Restrição

1. Existe um sistema controlado e um controlador

2. O controlador impõe restrições de segurança sobre a operação do sistema

3. Isso é feito emitindo ação de controle e observando o feedback do resultado.

4. O controle é um ciclo contínuo: ação → mudança de estado → feedback → nova ação, para garantir que as restrições sejam sempre cumpridas.

Este novo paradigma não diz que o que foi feito no passado está errado. A partir disso, as ideias tentam trazer uma perspectiva mais ampla.

O conceito de laço de controle é mais poderoso que a simples análise de falhas de componente. Ele permite capturar e explicar sistematicamente as causas de acidente complexos → Base do Modelo STAMP e também STPA.

STAMP

↳ Modelo de causalidade / acidente baseado em teoria de sistemas

As perdas são definidas: como problema de controle dinâmico ao invés de um problema baseado em falhas de componente.

↳ Laço de Controle. ↳ Estrutura de Controle Hierárquica.

• A aplicação é para sistemas complexos que não incluem apenas componente técnico, mas também software, design, fatores humanos.

O modelo STAMP: captura toda a conexão do modelo tradicional:

- Problema de interação entre componentes.
- Erros humanos
- Problema de software e requisitos
- Erros de design de sistema
- Fatores Organizacionais

O STAMP tem o laço de controle como elemento central.

Existem ações de controle potencialmente inseguras

↳ 1- Ações planejadas que causam perigo: o controlador emite uma ação que leva o sistema a um estado de risco.

↳ 2- Ações não planejadas que são necessárias: O controlador emite uma ação que ^{deixa de} seria ~~seria~~ essencial para controlar o processo com segurança.

Não basta identificar quais são inseguras, é fundamental identificar porque elas ocorrem.

Identificar ações de controle inseguras → Descobrir suas causas → Evitar mudanças e eliminar ou mitigar para evitar que o sistema atinja um estado de perigo e cause um acidente.

Uma estrutura de controle pode e geralmente deve incluir: Interação software com físico (Automático), Interação Humana com Automático (Software), Interação Humana com físico (direto), Interação humana com humana / social/organizacional para modelar adequadamente um sistema complexo.

Extensão é fundamental para analisar acidentes em sistemas
sistemáticos, onde o fator humano não pode ser reduzido a algoritmos.
O modelo original, do processo, possui um controlador dividido em duas partes, ~~processo~~
(modelo de processo + algoritmo de controle), porém isso é adequado para software,
porém não captura a complexidade da decisão humana.

1- Modelo Mental: visão interna do processo controlado e do ambiente ao redor.

↳ parecido com modelo de processo, porém mais rico, flexível e sujeito a erros.

2- Att modelo mental: processo contínuo que o humano recebe feedback e atualiza modelo mental.

↳ Problemas podem ocorrer se att for inadequada.

3- Seleção de ação de controle: processo de decisão final com base na topografia atual
do modelo mental e outras entradas (procedimento, pressão tempo, objetivo pessoal), e o humano
decide se irá tomar alguma decisão e se sim qual.

Complexidade abstrata: é impossível prever o comportamento humano.

Teoria do Sistema e Estrutura Hierárquica

• Cada nível hierárquico impõe restrições nas atividades do nível imediatamente abaixo.

↳ onde a ausência delas, causa comportamentos inadequados.

↳ A presença de controle ocorre entre os níveis da hierarquia para controlar a presença
na missão inferior.

Um acidente ocorre quando: a presença de controle fornece controle inadequado, e
as restrições de segurança são violadas no comportamento dos componentes no nível
de cliente.

Estrutura: Lado Esquerdo - Estrutura para desenvolvimento do sistema.

Lado Direito - Estrutura para operação do sistema.

↳ Segurança na operação depende fortemente de divisão e gestão de responsabilidades entre
diversos níveis e desenvolvimento dos sistemas, o que não ocorre nas interconexões.

O objetivo principal é criar uma estrutura que elimine ou reduza a possibilidade de
erro com uma definição clara de: responsabilidade (quem faz o quê), autoridade
(quem pode tomar quais decisões), ~~expectativa~~ (o que se espera de cada nível).

Complexidade e o uso de abstrações: estruturas de controle e segurança ^{extremamente} podem se tornar complexas.
↳ Para isso utilizamos abstrações, onde se oculta detalhes não necessários análise que está sendo desenvolvida
para a parte específica da

Os principais dados são primária abstrata em nível anterior e depois em top de nível, são
gerais as especificações → variáveis, segurança para o sistema e depois cada sub-sistema.

tilibra

Cast: Técnica usada para análise de causas de um acidente que já ocorreu.

STPA: Técnica usada para análise proativa de risco, identifica causas de acidentes antes do

Técnica mais simples

CAST = STPA e STPA-rec → Extensão Técnica STPA para aplicação em cibersegurança

Ter um bom modelo e uma técnica não é suficiente. Eles precisam ser incorporados em processos organizacionais e de engenharia para serem eficazes.

A STPA é superior as técnicas tradicionais no seguinte aspecto

1 - Identificar mais itens críticos, mais falhas de design e mais cenários de acidentes reais

2 - Aplicar STPA requer menos tempo e esforço que as técnicas tradicionais

STPA possui ROI positivo para pequenos/grandes projetos

4 - Passos da STPA

1 - Definir o propósito da análise.

↳ É o primeiro passo em qualquer análise com/sem métodos. // Qual tipo de perda a análise prevenir? Qual bagagem ^{esperada} ~~obtida~~ com as perdas identificadas, qual é o sistema a ser analisado. Deve aplicar-se apenas para safety qual ou prevenir ^{a perda da} vida humana. Ou se ele será aplicado mais amplamente aplicado para segurança, privacidade, performance ^{e outras} propriedades.

2 - Modelar o controle da estrutura.

↳ Captura o relacionamento funcional e interações, modelando o sistema e obtendo o feedback do control loop, começa em nível muito alto de abstração

3 - Identificar ações de controle inseguras.

↳ Analisar as control actions no controle da estrutura e examinar como lidam com a perda definida no 1º etapa

4 - Identificar cenários de perda.

↳ Identificar as razões pelo qual um controle inseguro pode ocorrer no sistema

Passo 1 - Definir o objetivo da análise

Identificar perdas

↳ "Perda" é algo que envolve algo de valor para o stakeholder.

O STPA pode ser usado para definir qual perda é inevitável para stakeholders.

↳ todo resultado do STPA vai ser mantido para uma ou mais perdas, sendo assim o resultado da análise pode ser facilmente resumido e priorizado.

Identificação de lugares em nível de sistema

Hazard: É um estado do sistema ou conjunto de condições que em conjunto com um conjunto particular de piores caso do ambiente levará a uma perda.

Primeiro, é necessário identificar o sistema a ser analisado e o limite do sistema. É para definir o limite do sistema devemos incluir a parte do sistema pelo qual projetistas tem controle. "O objetivo da engenharia de safety é mitigar/eliminar o hazard do sistema sob análise".

Critérios para definir o hazard

1- Hazard não ~~estado~~ ^{estado} de sistema ou condição, 2- Hazard vai levar a perda no pior caso do ambiente, 3- Hazard não estado ou condição a serem ~~prevenidas~~ ^{prevenidas}.

Dicas para prevenir erros na identificação de hazard.

• Hazard não devem se referir a componentes individuais do sistema. (devem se referir ao sistema num geral e ao estado do sistema) • hazard devem referir a falhas que podem ser controladas pela designer e operadores. • o hazard devem ser descritos em termos de condição a serem prevenidas.

De os hazard ocorrerem?

Constraint: podem definir como o sistema deve minimizar perdas caso o hazard ocorra.

Passo 2 - Modelar a estrutura de controle.

Estrutura de controle: É modelo sistema composto por laços de controle e feedback.

↳ Elementos: controller, control action, feedback, input and output from components, process controllers.

("Estrutura de controle não é uma estrutura física") e ("não é um modelo executável")

Definição de responsabilidades para cada controlador.

↳ Quando os controladores já foram identificados, a engenharia podem definir responsabilidades para eles.

A STPA assume que acidentes podem surgir por falhas em qualquer parte do loop.
↳ Problema no modelo do processo, problema de feedback, problema no algoritmo de controle.

A STPA usa abstração para controlar complexidade: ordem: componentes amplos, detalhes específicos

A lógica principal da STPA é: identificar controladores, refinar subsistema, definir responsabilidades, derivar control action, derivar feedback, refinar iterativamente

Passo 3 - Identificar ações de controle inseguras

UCA - Ação de controle insegura: É uma ação de controle que em um contexto particular e no pior caso do ambiente leva a um hazard.

UCA não fornecida: a ação deveria ocorrer, mas não ocorreu

UCA fornecida: a ação ocorre quando não deveria ↳ <duracão errada>

UCA sobtuda...: problema temporal. Aplicada por muito tempo interrompida etc: problema de duração ↳ <momento errado>

Exemplo	Source	control action	Link to hazards
UCA-2:	BSCU Autobrake	Brake command	during a normal takeoff [H-1.3]
	Type		context

Uma UCA que ocorreu não garante que SEMPRE ocorrerá um HAZARD.

Subsequências não eliminam UCAs, (mesmo existindo backup, redundância, sistema próximo) a UCA continua válida, porque subsequências podem falhar, STPA analysis é Pior Caso

Uma categoria pode ter qta várias UCAs.

Toda UCA deve estar ligada a um hazard, caso for identificada esse caso: provavelmente falhou identificação um hazard. ↳ por manutenção.

Não deve-se confundir contexto com causa, Contexto: Estado RQL do sistema

Causa: O controlador acreditou nisso

Dempre: especificar contexto, usar estados reais, definir UCAs claramente, manter rastreabilidade com hazard, revisar caso "não otimizado"

Restrição é o universo UCA.

Passo 1: identificar cenários de perda.

Cenário de Perda: ~~Identificar~~ Descreve fator causador, condição, falha... que podem levar a ação de controle insegura e a perda.

Cenários que levam a UCLs ("Porque o controlador tomou uma decisão insegura?")

↳ Comportamento inseguro do controlador

↳ Falhas físicas do computador, algoritmos de controle inadequados (software tomou decisão errada), entrada insegura de outro controlador (controlador recebe comando incorreto de outro componente), modelo de processo inadequado (controlador mantém um "modelo mental" do sistema, se esse modelo estiver errado, ele toma decisão errada)

STPA também pode tratar: ataques, invasões, adulteração, spoofing, negação de serviço

↳ Um adversário também pode causar perda.

2. Cenários onde a ação de controle é executada inadequadamente

↳ "O controlador mandou fazer algo errado, mas porque isso não funcionou?"

↳ Problema no caminho de controle: Controlador, atuadora, comunicação, sensor, real-veler, restora

↳ Problema no processo controlado: Ambiente ou processo físico impede o efeito esperado

Não identificar apenas falhas isoladas: STPA quer: cenários completos, interações, contextos, combinação de fatores, FOCAR SÓ EM FALHA INDIVIDUAL transforma a análise em algo parecido com Failure Mode and Effect Analysis Não é o objetivo da STPA.

No STPA tudo deve ser rastreável: Perda → Hazard → UCL → Cenário de Perda, assim conseguimos justificar requisitos, explicar decisões, entender causas, revisar arquitetura e validar segurança.

Os resultados do STPA podem ser usados para definir requisitos, melhorar arquitetura, criar subcomponentes, gerar caso de teste, orientar design, identificar riscos, melhorar sistema de gerenciamento.

Semana 8

- A abertura é de mão única. - Controle automático de fechamento da porta quando o trem está aproximando e ~~para~~ e abre quando ele passa. - Sensor capaz de identificar se o trem tá próximo ou não. - Existem apenas sinais de luz, mas não sonora.

1.1 Definir e delimitar o problema.

O problema é que basicamente 400 pessoas morrem na estação e 300 ANo.

1.2 Definir o propósito da análise

1.2 Componentes

- Road, Railway, Crosswalk, light signal, sensor, controle portão.

1.3 Stakeholders

- Motorista trem, motorista veículo, pedestre, dono da railway, e dono. sempre

1.4 System purpose and goal

SG-1 - Permitir passagem segura trem, carro and humano.

SG-2 - Permitir que o trajeto do trem não sofra impedimento

SG-3 - Evitar danos ao veículo envolvido e pedestre

SG-4 - Ter controle de abertura e fechamento no momento ideal

1.5 Identify losses and accident

L1 - Perda de vidas humanas

L2 - Danos a abertura, impedindo por um tempo locomotiva no vi

L3 - Danos ao veículo

L4 - Perda econômica

1.6.1 Identify and map system hazards and system level constraints

H1 - Carro/pedestre/animal na railway L1 L2 L3 L4

H2 - Má sinalização ou malfuncionamento L1 L2 L3 L4

H3 - Sensor com falha relacionada pedestre L1 L2 L3 L4

H4 - Trem parado muito tempo L1 L2 L3 L4

1.6.2 Identify and map system safety constraints to system hazards

H1 → Não podem cruzar quando a porta está aberta (segura)

H2 → Devem estar constantemente incluídos em locais visíveis e principalmente com sinalização

H3 → Deve-se verificar se o sensor de proximidade está correto

H4 → As medidas de segurança para evitar acidentes no sentido de

evitar bem